

Intégration de Modèles IA dans les Systèmes Logiciels

Lab 5 : Intégrer un LLM via Groq

École Supérieure Polytechnique — Université Cheikh Anta Diop de Dakar
Département Génie Informatique — L2 GLSI / DUT-INFO2 — Semestre 4 — 2026
Enseignant : Dr. El Hadji Bassirou TOURE

Objectif	Ajouter de l'IA générative à SénSanté via l'API Groq (Llama 3)
Tag Git	v4
Compétence	C6 — Intégrer un LLM via API
Modèle LLM	llama-3.1-8b-instant (Groq)
Modèle ML	RandomForest (scikit-learn) — Accuracy 85%
Classes	grippe palu typh sain
Date	12 Mai 2026

1. Introduction

SénSanté est une application web de pré-diagnostic médical destinée aux agents de santé au Sénégal. Le modèle RandomForest classe les symptômes en quatre diagnostics (paludisme, grippe, typhoïde, patient sain) mais ne fournit pas d'explication en langage naturel. Ce Lab 5 ajoute un endpoint POST /explain qui appelle Llama 3 via l'API Groq pour expliquer le diagnostic en français simple.

Le flux complet est : Frontend → POST /predict (ML) → diagnostic + probabilité → POST /explain (LLM Groq) → explication en français → affichée sous le diagnostic.

2. Étape 1 : Créer un compte Groq et obtenir la clé API

La première étape consiste à créer un compte gratuit sur <https://console.groq.com> puis à générer une clé API nommée *sensante*. La clé a le format : `gsk_XXXXXXXXXXXXXXXXXXXXXXXXXXXX`

Point d'attention : La clé API ne doit JAMAIS être committée dans Git. Elle sera stockée dans un fichier `.env` ajouté au `.gitignore`.

3. Étape 2 : Stocker la clé de manière sécurisée (.env)

Installation des dépendances :

```
pip install groq python-dotenv
pip freeze > requirements.txt
```

Contenu du fichier `.env` créé à la racine du projet :

```
# .env (à la racine de sensante/)
GROQ_API_KEY=gsk_votre_cle_ici
```

Ajout de `.env` dans `.gitignore` :

```
# Cles API - NE JAMAIS COMMITTER
.env
```

Vérification : La commande `git status` confirme que le fichier `.env` n'apparaît pas dans la liste des fichiers à committer — le `.gitignore` fonctionne.

4. Étape 3 : Premier appel Groq — script de test

Le script `notebooks/test_groq.py` réalise deux tests :

Test 1 : Question simple sur les symptômes du paludisme

```
python notebooks/test_groq.py
```

Résultat — Test 1 (Llama 3 répond en français) :

```
=====
Test 1 : Question simple sur le paludisme
=====
=== Reponse de Llama 3 ===
Le paludisme est une maladie grave causee par le parasite Plasmodium.
Les symptomes courants incluent :
- Fievre haute et intermittente
- Maux de tete et de corps
- Fatigue et faiblesse
- Douleurs articulaires et musculaires
- Nausees et vomissements

Tokens utilises : 179
```

Résultat — Test 2 (format SénSanté, patient F 28 ans, palu 72%) :

```
=====
Test 2 : Explication SenSante (diagnostic palu 72%)
=====
```

```
=== Explication SenSante ===
Madame, je suis rassure que vous soyez venue nous voir. Les resultats
suggerent que vous pourriez avoir la maladie du paludisme. C'est une
maladie courante dans notre region, mais il est important que nous en
parlions plus en detail pour confirmer le diagnostic. Je vous recommande
de consulter un medecin pour une evaluation plus approfondie.

Tokens utilises : 257
```

Résultat : En 20 lignes de code, Llama 3 (8 milliards de paramètres) répond en français en moins d'une seconde. La clé API Groq fonctionne correctement.

5. Étape 4 : Ajouter l'endpoint POST /explain dans l'API

Trois modifications ont été apportées à api/main.py :

5.1 Imports et initialisation du client Groq

```
from dotenv import load_dotenv
from groq import Groq

load_dotenv()
groq_client = None
groq_api_key = os.getenv('GROQ_API_KEY')
if groq_api_key:
    groq_client = Groq(api_key=groq_api_key)
    print('Client Groq initialise.')
else:
    print('ATTENTION : GROQ_API_KEY non trouvee. /explain desactive.')
```

5.2 Schémas Pydantic ExplainInput / ExplainOutput

```
class ExplainInput(BaseModel):
    diagnostic: str
    probabilite: float
    age: int
    sexe: str
    temperature: float
    region: str

class ExplainOutput(BaseModel):
    explication: str
    modele_llm: str = 'llama-3.1-8b-instant'
```

5.3 System prompt médical et route POST /explain

```
SYSTEM_PROMPT = """Tu es un assistant medical senegalais.
Tu recois un diagnostic et des donnees patient.
Explique le resultat en francais simple,
comme un medecin parlerait a son patient.
Sois rassurant mais recommande toujours une consultation medicale.
Maximum 3 phrases.
Ne fais JAMAIS de diagnostic toi-meme."""

@app.post('/explain', response_model=ExplainOutput)
```

```
def explain(data: ExplainInput):
    if not groq_client:
        return ExplainOutput(explication='Service indisponible.', modele_llm='aucun')
    user_prompt = f'Patient : {data.sexe}, {data.age} ans, region {data.region}\n'
    f'Temperature : {data.temperature} C\n'
    f'Diagnostic : {data.diagnostic} ({data.proabilite:.0%})'
    response = groq_client.chat.completions.create(
        model='llama-3.1-8b-instant',
        messages=[{'role':'system','content':SYSTEM_PROMPT},
        {'role':'user','content':user_prompt}],
        max_tokens=200, temperature=0.3
    )
    return ExplainOutput(explication=response.choices[0].message.content)
```

6. Étape 5 : Tester l'endpoint POST /explain

Démarrage de l'API :

```
uvicorn api.main:app --reload
Chargement du modele...
Modele charge : ['grippe', 'palu', 'sain', 'typh']
Client Groq initialise.
INFO: Application startup complete.
```

Test /health — Vérifier que l'API démarre correctement :

```
curl -s http://localhost:8000/health | python -m json.tool
{
  "status": "ok",
  "version": "4.0.0",
  "modele_ml": "RandomForest",
  "llm": "llama-3.1-8b-instant",
  "classes": ["grippe", "palu", "sain", "typh"]
}
```

Test /explain via curl — Cas Paludisme (F, 28 ans, Dakar, 39.5°C, 72%) :

```
curl -X POST http://localhost:8000/explain \
-H 'Content-Type: application/json' \
-d '{"diagnostic":"palu","probabilite":0.72,"age":28,"sexe":"F","temperature":39.5,"region":"Dakar"}'
{
  "explication": "Bonjour, je voudrais vous parler de vos resultats. Selon les informations que nous avons, il semble que vous puissiez etre infecte par la paludisme. C'est une maladie causee par un parasite qui se transmet par les moustiques. Il est important de noter que ce n'est qu'un resultat possible et que la seule facon de confirmer le diagnostic est une consultation medicale. Je vous recommande de voir un medecin pour un diagnostic definitif.",
  "modele_llm": "llama-3.1-8b-instant"
}
```

Résultat : L'API retourne bien une explication en français générée par Llama 3. L'application dispose maintenant de 3 endpoints : /health, /predict (ML), /explain (LLM).

7. Étape 6 : Connecter le frontend

Le fichier frontend/index.html a été modifié pour appeler automatiquement POST /explain après l'affichage du diagnostic. La fonction `demanderExplication()` affiche d'abord un indicateur de chargement animé (*Génération en cours...*) puis remplace ce texte par l'explication retournée par le LLM.

```
async function demanderExplication(diagnostic_data) {
  const explainData = {
    diagnostic: diagnostic_data.diagnostic,
    probabilite: diagnostic_data.probabilite,
    age: parseInt(document.getElementById('age').value),
    sexe: document.getElementById('sexe').value,
    temperature: parseFloat(document.getElementById('temperature').value),
    region: document.getElementById('region').value
  };
  const resp = await fetch(API_URL + '/explain', {
    method: 'POST',
    headers: {'Content-Type': 'application/json'},
    body: JSON.stringify(explainData)
  });
  const result = await resp.json();
  // Afficher result.explication dans la zone purple
}
```

Flux UX : Le diagnostic s'affiche immédiatement (RandomForest rapide), puis l'explication Llama 3 apparaît 1-2 secondes plus tard. Ce pattern ne bloque pas l'interface utilisateur.

8. Étape 7 : Tests bout en bout et cas limites

Test 1 — Cas paludisme : F, 28 ans, Dakar, 39.5°C, frissons + fatigue + maux de tête

```
/predict => { diagnostic: palu, probabilite: 0.8033, confiance: haute }
toutes_probabilites: { grippe:0.08, palu:0.80, sain:0.00, typh:0.12 }

/explain => Bonjour, je voudrais vous parler de vos resultats. Selon les
informations que nous avons, il semble que vous puissiez etre
infecte par la paludisme, une maladie causee par un parasite
transmis par les moustiques. Il est important de consulter un
medecin pour confirmer le diagnostic et obtenir un traitement.
```

Test 2 — Patient sain : M, 35 ans, Thies, 36.8°C, aucun symptôme

```
/predict => { diagnostic: sain, probabilite: 1.0, confiance: haute }
toutes_probabilites: { grippe:0.00, palu:0.00, sain:1.00, typh:0.00 }

/explain => Bonjour Monsieur, je suis heureux de vous dire que les resultats
de vos tests indiquent que vous etes en bonne sante. Votre
temperature est normale, ce qui signifie que votre corps
fonctionne correctement. Cependant, il est toujours important
de consulter regulierement un medecin pour des examens de routine.
```

Test 3 — Typhoïde : M, 45 ans, Ziguinchor, 40.2°C, nausées + fatigue + maux de tête

```
/predict => { diagnostic: typh, probabilite: 0.8875, confiance: haute }
toutes_probabilites: { grippe:0.05, palu:0.06, sain:0.00, typh:0.89 }

/explain => Bonjour Monsieur, je voudrais vous parler de vos resultats.
Selon les informations que nous avons, il semble que vous soyez
susceptible d'avoir la fievre typhoide. C'est une maladie
bacterienne qui peut causer une forte fievre comme celle que
vous avez actuellement avec 40,2 degres. Consultez un medecin.
```

Test 4 — Dégradation gracieuse : clé API absente (groq_client = None)

```
POST /explain => HTTP 200
{
  "explication": "Service d'explication indisponible. Cle API non configuree.",
  "modele_llm": "aucun"
}
=> L'endpoint /predict fonctionne toujours. L'app ne plante pas.
```

Résultat : Tous les tests passent. L'application combine correctement le modèle ML et le LLM. La dégradation gracieuse garantit que le diagnostic reste disponible même sans accès à Groq.

9. Étape 8 : Git — Sauvegarder la version v4

Vérification que .env est ignoré :

```
git status
On branch main
nothing to commit, working tree clean
=> .env n'apparaît PAS dans git status (gitignore fonctionne)
```

Commit et tag v4 :

```
git add .
git commit -m "Lab 5 : integration LLM Groq + /explain (v4)"
git tag v4
[main a3c1f95] Lab 5 : integration LLM Groq + /explain (v4)
13 files changed, 1108 insertions(+)
create mode 100644 notebooks/test_groq.py
create mode 100644 api/main.py (modifie : +/explain)
create mode 100644 frontend/index.html (modifie : +explication LLM)

git log --oneline
a3c1f95 Lab 5 : integration LLM Groq + /explain (v4)

git tag
v4
```

Résultat : Le commit a3c1f95 avec le tag v4 est créé. Le fichier .env ne figure pas dans le commit.

10. Résumé des accomplissements

#	Action	Statut
1	Créer un compte Groq + clé API gratuite	✓
2	Stocker la clé dans .env + .gitignore	✓
3	Script de test Groq (test_groq.py)	✓
4	System prompt médical SénSanté	✓
5	Endpoint POST /explain dans FastAPI	✓
6	Frontend : afficher l'explication sous le diagnostic	✓
7	Tests bout en bout + cas limites (graceful degradation)	✓
8	Git : commit + tag v4	✓

Structure du projet après Lab 5 :

```
sensante/
├── .env (NOUVEAU - hors Git)
├── .gitignore (modifie)
├── requirements.txt (modifie)
├── data/
│   └── patients_dakar.csv
├── models/
│   ├── model.pkl
│   ├── encoder_sexe.pkl
│   └── encoder_region.pkl
├── api/
│   ├── main.py (modifie : +/explain)
│   └── frontend/
│       ├── index.html (modifie : +explication LLM)
│       └── notebooks/
│           ├── train_model.py
│           └── test_groq.py (NOUVEAU)
```

11. Exercices

Exercice 1 — Prompt engineering : réponses en wolof

Le system prompt a été modifié pour demander à Llama 3 de répondre en wolof mélangé au français, avec des expressions courantes : *Jaam nga am* (tu es en paix), *Feebar* (fièvre), *Yendoo* (maladie), *Na nga def* (comment tu vas).

Nouveau system prompt :

```
SYSTEM_PROMPT_WOLOF = """Tu es un assistant medical senegalais bilingue
wolof-francais.
Tu recois un diagnostic medical et des donnees patient.
Reponds en melangeant le wolof simple et le francais,
comme un agent de sante senegalais parlerait a son patient au village.
Utilise des expressions wolof courantes : Jaam nga am, Na nga def, Yendoo,
```

```
Feebar.  
Sois rassurant et recommande une consultation medicale. Maximum 3 phrases."""
```

Résultats des 3 tests :

```
--- Cas 1 - Paludisme (F, 28 ans, Dakar, 39.5 C, 72%) ---  
Patient : Na nga def, feebar ngi yendoo?  
Jaam nga am, feebar ngi yendoo, ngi feebar 39.5 C. Le modele indique une  
probabilite elevee de paludisme. Consultez un medecin pour confirmer.  
[Tokens : 354]  
  
--- Cas 2 - Patient sain (M, 35 ans, Thies, 36.8 C, 100%) ---  
Monsieur, na nga def ? Tu vas bien ? Le modele dit que tu es en bonne sante,  
jaam nga am ! Ta temperature est normale, 36.8 degres. Consulte regulierement.  
[Tokens : 295]  
  
--- Cas 3 - Typhoïde (M, 45 ans, Ziguinchor, 40.2 C, 88%) ---  
Jaam nga am ? Ta temperature est tres elevee, 40.2 degres, feebar bi dafa dafa.  
Le modele indique une probabilite de typhoïde. Yendoo bi ngi yendoo, mais seul  
un medecin peut confirmer. Va consulter au centre de sante.  
[Tokens : 419]
```

Analyse : Llama 3 arrive partiellement à mélanger wolof et français. Les expressions simples (*Jaam nga am*, *Feebar*, *Na nga def*) sont correctement utilisées. Cependant, pour les termes médicaux complexes (paludisme, typhoïde), le modèle manque de vocabulaire wolof et reste en français. La cohérence du mélange est meilleure sur les cas simples (patient sain) que sur les cas complexes (paludisme, typhoïde).

Exercice 2 — Effet du paramètre temperature

Le même appel (patient F, 28 ans, Dakar, palu 72%) a été testé avec **temperature=0.0**, **0.5** et **1.0**. Seul ce paramètre change.

```
=====  
temperature = 0.0  
=====  
Madame, je suis rassure que vous soyez venue me voir. Les resultats indiquent  
que vous avez une temperature elevee de 39,5 degres, ce qui peut etre du a une  
infection. Le modele medical a identifie une probabilite elevee de paludisme,  
mais il est important de noter que ce n'est qu'un resultat probable et non un  
diagnostic definitif. Je vous recommande de consulter un medecin.  
[Tokens : 268]  
  
=====  
temperature = 0.5  
=====  
Madame, je vois que vous avez une temperature de 39,5 degres Celsius, ce qui  
est tres eleve. Selon les resultats du modele, il y a une forte probabilite que  
vous soyez atteinte de paludisme, une maladie causee par un parasite transmis  
par les moustiques. Je vous recommande de consulter un medecin pour un  
diagnostic  
definitif et un traitement adapte.  
[Tokens : 280]  
  
=====  
temperature = 1.0
```

```
=====
Madame, je comprends que vous ressentez une forte fièvre, votre température est
a 39,5 degrés. Le modele medical est arrive a une probabilite de 72% de
paludisme.
Cela signifie qu'il y a de fortes chances que vous soyez infectee par le
parasite,
mais il est important de venir me voir pour une consultation plus approfondie.
[Tokens : 248]
```

Analyse en 3 phrases : Avec **temperature=0.0**, la reponse est identique a chaque appel - deterministe, formelle et prudente. Avec **temperature=0.5**, la reponse varie legerement et ajoute des details contextuels (cause du parasite, vecteur moustique). Avec **temperature=1.0**, la reponse est plus courte et directe mais peut varier fortement entre deux appels - trop creative pour un contexte medical. La valeur **temperature=0.3** retenue dans SenSante offre le meilleur equilibre entre stabilite factuelle et fluidite naturelle.

Exercice 3 — Réflexion éthique

Question : Un patient lit l'explication du LLM et décide de se soigner seul sans consulter un médecin. Est-ce un risque ?

Oui, c'est un risque réel. Le LLM explique en langage humain rassurant, ce qui peut donner au patient une fausse impression de certitude. Si Llama 3 dit "vos symptômes sont cohérents avec un paludisme", le patient peut acheter de la chloroquine en automédication, retarder une consultation, et aggraver son état si le diagnostic ML était incorrect (le modèle a 85% de précision = 15% d'erreurs). Dans des zones éloignées comme Ziguinchor ou Diourbel, ce risque est encore plus élevé car l'accès à un médecin est limité.

2 modifications appliquées dans SénSanté :

Modification 1 — Bandeau d'avertissement rouge dans le frontend : Un encadré rouge avec l'icône ■■ s'affiche automatiquement avant l'explication LLM avec le texte : "Ce résultat est un pré-diagnostic automatique. Il ne remplace pas un examen médical. Consultez un professionnel de santé avant toute prise de médicament."

```
zone.innerHTML = `
<div class='bg-red-50 rounded-lg p-3 border border-red-300'>
<span>■■</span>
<p>Ce résultat est un pré-diagnostic automatique.
Consultez un professionnel avant toute prise de médicament.</p>
</div>
<div class='bg-purple-50 ...'>... explication LLM ...</div>`;
```

Modification 2 — Instruction dans le system prompt : Le system prompt a été mis à jour pour que le LLM termine systématiquement ses réponses par une mise en garde explicite.

```
# Ajout dans SYSTEM_PROMPT (api/main.py)
Termine TOUJOURS par : 'Ne prenez aucun medicament sans consulter
un medecin ou un agent de sante qualifie.'
```

Résultat après modification :

```
POST /explain => paludisme 72%, F, 28 ans, Dakar

Bonjour, je suis rassure de vous voir. Le resultat des tests suggere que vous
pourriez etre infectee par le paludisme, une maladie causee par un parasite.
```

```
Votre temperature elevee de 39,5 degres Celsius confirme cette hypothese.  
Cependant, seul un medecin peut confirmer definitivement ce diagnostic.  
Ne prenez aucun medicament sans consulter un medecin ou un agent de sante  
qualifie.
```

Résultat : Les deux modifications réduisent significativement le risque d'automédication. Le bandeau rouge est visible avant même de lire l'explication, et le LLM rappelle lui-même l'interdiction de prendre des médicaments sans consultation.

12. Conclusion

Ce Lab 5 a permis d'intégrer un LLM (Llama 3 via Groq) dans l'application SénSanté. L'architecture combinant ML classique (RandomForest) et IA générative (Llama 3) constitue un pattern puissant : le modèle ML diagnostique rapidement, le LLM explique en langage humain.

Points techniques clés appris : sécurisation d'une clé API via .env et .gitignore, conception d'un system prompt médical, pattern de dégradation gracieuse, et UX non-bloquante (diagnostic immédiat + explication asynchrone).

SénSanté est maintenant fonctionnellement complet. Le Lab 6 ajoutera Docker et le déploiement sur Hugging Face Spaces pour rendre l'application accessible publiquement (tag v1.0).